

DataFlow-Harness: Editable LLM Data Pipeline 구성을 위한 Grounded Code-Agent Platform

Runming He*, Zhen Hao Wong*, Hao Liang^{*,†}, Zimo Meng*, Chengyu Shen,
Xiaochen Ma, Wentao Zhang[‡]

¹Peking University, ²Institute for Advanced Algorithms Research, Shanghai,
³Zhongguancun Academy

Large language models (LLMs)는 data-processing workflow 자동화에 점점 더 많이 사용되고 있지만, coding agent가 생성한 script는 일반적으로 persistent하고 editable한 platform artifact로 자동 materialize되지 않는다. 우리는 이 disconnect를 *NL2Pipeline gap*이라고 부른다. 이를 해소하기 위해 DataFlow-Harness를 제안한다. 이 platform은 free-form script 대신 typed, incremental mutation을 통해 LLM agent가 platform-native directed acyclic graph (DAG)를 구성하도록 안내한다. Platform은 procedural guidance를 제공하는 DataFlow-Skills, live operator registry와 현재 pipeline state를 노출하는 Model Context Protocol (MCP) layer, conversational authoring과 visual DAG editor를 동기화하는 DataFlow-WebUI를 결합한다. 12개 task로 구성된 data-engineering benchmark에서 DataFlow-Harness는 관찰된 end-to-end pass rate 93.3%를 달성한다. Vanilla Claude Code와 비교하면 측정된 monetary cost를 72.5%, generation latency를 49.9% 줄인다. Context-Aware Claude Code baseline과 비교하면 관찰된 pass rate는 0.9 percentage point 이내이며, cost는 42.8% 더 낮다. Task별 분석은 construction이 암묵적 procedural knowledge에 의존할 때 Skills가 가장 유용함을 보여준다. 이러한 결과는 live platform grounding이 script-generation baseline에 가까운 관찰 reliability를 유지하면서도 더 낮은 measured construction cost와 latency로 persistent하고 editable한 workflow artifact를 생성할 수 있음을 보여준다.

*Equal Contribution, [†]Project Leader, [‡]Corresponding Author

✉ Correspondence : wentao.zhang@pku.edu.cn

🔗 Source Code : <https://github.com/OpenDCAI/DataFlow-WebUI>

Codebase Documentation : <https://opendcai.github.io/DataFlow-Doc/>

Contents

1	서론	3
2	관련 연구	3
2.1	Code Generation을 위한 Agent.	3
2.2	Data Engineering과 LLM Pipeline.	4
2.3	LLM 기반 Workflow Synthesis.	4
3	시스템 아키텍처	4
3.1	Data Pipeline Backend	5
3.2	DataFlow-WebUI	5
3.3	MCP Tools Layer	6
3.4	DataFlow-Skills	6
4	실험	6
4.1	실험 설정	7
4.2	Workflow Synthesis 효과 (RQ1)	7
4.3	Efficiency와 System Cost (RQ2)	8
4.4	Textbook-to-VQA Workflow 평가	8
4.5	Ablation과 Micro-mechanisms (RQ3)	9
4.6	Downstream Training Utility Evaluation (RQ4)	10
5	Conclusion	12

1 서론

대규모 언어 모델(LLM)은 합성 데이터 생성, 평가, 검색 증강, 모델 학습과 같은 애플리케이션을 위한 데이터 처리 workflow를 구성하는 데 점점 더 많이 배포되고 있다 [9–11]. 최근의 coding agent는 자연어 요구사항을 실행 가능한 구현으로 자동 변환할 수 있어, 이러한 workflow를 구성하는 데 필요한 노력을 크게 줄인다 [7, 25].

그러나 높은 task accuracy만으로는 production deployment에 충분하지 않은 경우가 많다. 산업 환경에서 workflow artifact는 생애주기 전반에 걸쳐 가시적이고, 편집 가능하며, 재사용 가능하고, 플랫폼 governance 메커니즘과 호환되어야 한다 [12]. 우리의 경험상 직접 code-generation agent는 source code로만 존재하는 일회용 script를 자주 생성한다. 이러한 출력은 graphical workflow interface를 통해 audit하기 어렵고 종종 dependency를 환각한다 [19, 22]. 즉 사용 불가능한 operator, 오래된 플랫폼 가정, 또는 범용 agent가 추론하기 어려운 framework-specific 동작에 의존한다 [18, 20, 21].

우리는 이 문제를 *NL2Pipeline gap*으로 정의한다. 사용자는 workflow 요구사항을 자연어로 표현하지만, production environment는 시각화, 편집, 재사용이 가능한 구조화되고 지속적인 pipeline asset을 요구한다. 여기서 workflow는 의도된 데이터 처리 절차를, pipeline representation은 그 지속적인 플랫폼 객체를, DAG는 실행 dependency를 나타낸다. 이 gap을 해소하려면 code-generation accuracy를 개선하는 것 이상이 필요하다. 구성 과정은 플랫폼 semantics에 grounded되어야 하며 host platform과 통합되는 artifact를 생성해야 한다.

이 한계를 해결하기 위해 우리는 grounded workflow synthesis를 위한 플랫폼인 DataFlow-Harness를 제안한다. DataFlow-Harness는 script를 직접 생성하는 대신 세 가지 분리된 구성요소를 통해 agent가 platform-native workflow를 구성하도록 안내한다. DataFlow-Skills는 operator-selection pattern, schema dependency, assembly procedure를 포함한 domain-specific 구성 지식을 encode한다. Model Context Protocol (MCP)은 live operator registry와 현재 workflow state에 대한 access를 제공하여 agent action을 execution environment에 grounding한다 [2]. 마지막으로 DataFlow-WebUI는 iterative refinement를 위한 conversational interface를 제공하고, 생성된 workflow를 지속적이고 편집 가능한 visual DAG로 materialize한다.

우리는 데이터 변환, question answering, quality filtering, synthetic data generation을 포괄하는 pipeline-construction task에서 DataFlow-Harness를 평가한다. 관측된 end-to-end pass rate는 이 benchmark에서 script-generation baseline에 가깝지만, 측정된 token usage, cost, latency는 더 낮다.

우리의 contribution은 다음과 같다.

- 우리는 *NL2Pipeline gap*을 정식화한다. 이는 자연어 workflow intent와, inspection 및 editing을 위해 계속 이용 가능한 지속적이고 platform-native인 workflow artifact 사이의 단절이다.
- 우리는 procedural Skills, live MCP grounding, typed incremental mutation, validation, 그리고 동기화된 conversational 및 visual authoring interface를 결합한 DataFlow-Harness를 제시한다.
- 우리는 12-task benchmark에서 reliability와 construction efficiency를 평가하고, per-task ablation을 통해 Skills가 도움이 되는 지점을 분석하며, downstream training utility에 대한 두 가지 controlled case study를 제공한다.

2 관련 연구

2.1 Code Generation을 위한 Agent.

Code synthesis는 Codex [5]와 StarCoder [15] 같은 foundational pretrained model에서 autonomous agentic loop로 발전해 왔다. Reflexion [23]과 Self-Debug [6]는 environment feedback을 통한 iterative refinement를 도입했다. MCP [2]는 agent를 위한 unified tool interface를 제공한다. SWE-agent [26]와 Claude Code [1] 같은 현재 agent는 repository-level multi-turn editing과 verification에 초점을 둔다. 우리의 연구는 agent 자체를 개선하기보다 domain-specific harness 안에서 general-purpose agent를 제약한다는 점에서 다르다.

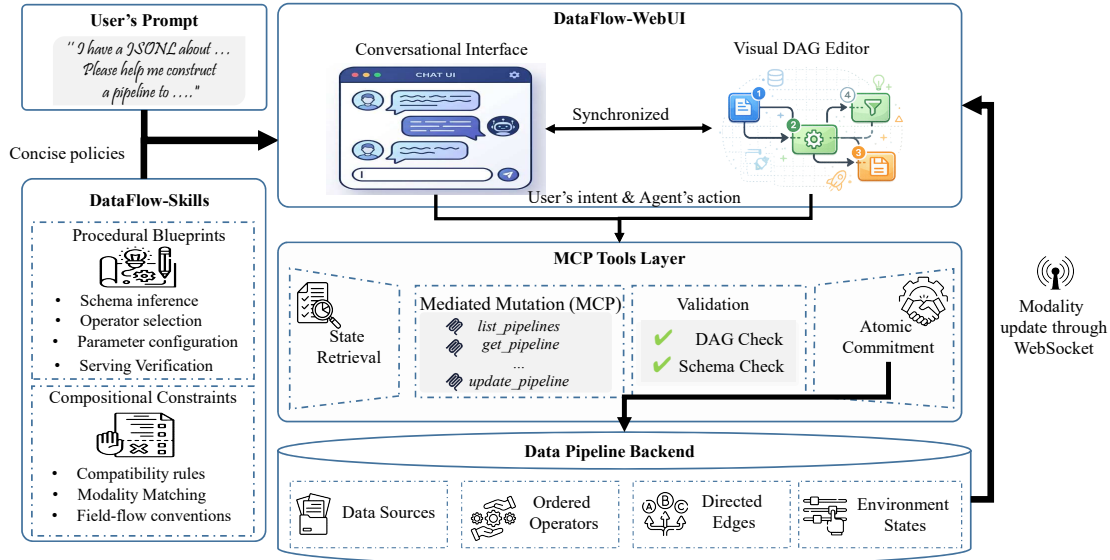


Figure 1 DataFlow-Harness architecture. 공유 pipeline representation은 agent runtime과 DataFlow-WebUI 전반에서 동기화된다. DataFlow-Skills는 construction을 안내하고, Validation Engine은 DAG structure와 schema compatibility를 검사한다.

2.2 Data Engineering과 LLM Pipeline.

Data-centric AI [28]는 model architecture보다 systematic governance를 강조한다. Data-Juicer [4]와 DCLM [14] 같은 specialized system은 extensible operator를 통해 large-scale curation을 operationalize한다. DataFlow [16]와 DSPy [11]는 LLM operation을 formal execution graph 안의 composable component로 다룬다. 우리의 연구는 DataFlow를 기반으로 하지만, pipeline execution이 아니라 pipeline 자체의 agent-assisted construction에 초점을 둔다.

2.3 LLM 기반 Workflow Synthesis.

최근 시스템은 standalone program이 아니라 structured workflow를 생성한다. AutoFlow는 LLM agent를 위한 reusable workflow를 자동으로 synthesize하는 반면 [13], Balis et al.은 과학 연구 질문을 validated workflow DAG로 변환하고 domain knowledge를 reusable Skills로 encode한다 [3]. 이러한 노력은 structured workflow generation의 가치를 확립한다. DataFlow-Harness는 live data-engineering platform 내부의 interactive, stateful authoring이라는 상호보완적 systems problem에 초점을 맞춘다. agent는 MCP를 통해 현재 pipeline과 operator registry를 retrieve하고, 기존 artifact에 typed incremental mutation을 적용하며, 제안된 각 state를 validate하고, conversational interface 및 visual editor와 하나의 persistent representation을 공유한다. 따라서 우리의 contribution은 단순한 NL-to-DAG generation이 아니라, live workflow artifact의 platform-grounded construction과 iterative editing이다.

3 시스템 아키텍처

DataFlow-Harness는 workflow synthesis를 네 가지 구성요소, 즉 DataFlow-WebUI, MCP Tools Layer, Data Pipeline Backend, DataFlow-Skills를 중심으로 조직한다.

operational lifecycle은 conversational, visual, programmatic interface 전반에서 authoritative source of truth 역할을 하는 Data Pipeline Backend를 중심으로 한다. Pipeline mutation은 MCP Tools Layer를 통해 발행되고, validation을 거쳐 backend에 commit된 다음 DataFlow-WebUI와 동기화된다. 이와 병행하여 DataFlow-Skills는 pipeline state를 직접 수정하지 않으면서 agent reasoning을 형성하는 procedural guidance를 제공한다.

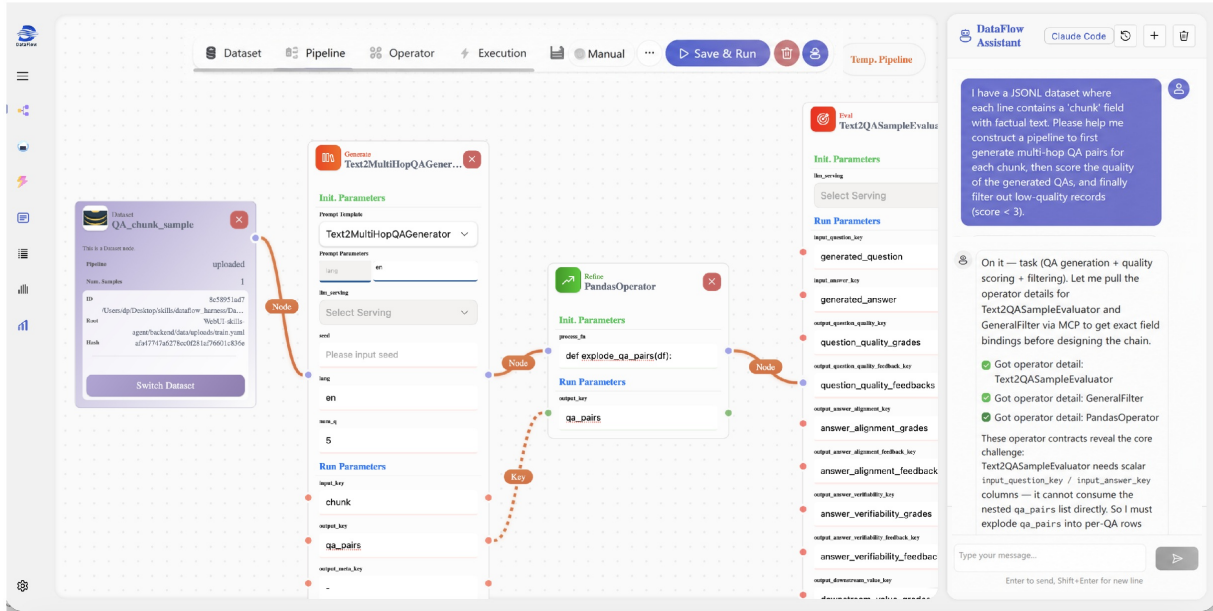


Figure 2 DataFlow-WebUI의 dual-modality interface. conversational agent와 visual DAG editor 사이의 synchronization을 보여 준다.

3.1 Data Pipeline Backend

Data Pipeline Backend는 workflow synthesis를 위한 authoritative source of truth 역할을 한다. 우리는 pipeline을 $P = (D, O, E, S, R)$ 로 표현한다. 여기서 D 는 data source와 그 URI의 집합, O 는 configured operator instance의 집합, $E \subseteq O \times O$ 는 directed data-dependency edge를 포함하며, S 는 input 및 output field schema를 기록하고, R 은 model-serving endpoint와 같은 runtime state를 포함한다.

free-form code를 생성하는 대신, agent는 operator 추가 또는 제거, parameter update, edge 연결을 포함한 typed mutation을 통해 backend와 상호작용한다. mutation은 결과 graph가 acyclic이고 인접 operator schema가 compatible한 경우에만 commit된다. 이러한 check는 structural validity를 확립하지만, 그 자체로 semantic correctness, endpoint availability, 또는 output quality를 보장하지는 않는다.

3.2 DataFlow-WebUI

DataFlow-WebUI는 workflow construction을 위한 두 가지 동기화된 modality를 제공한다. 하나는 natural-language authoring을 위한 conversational interface이고, 다른 하나는 직접적인 workflow inspection 및 editing을 위한 visual DAG editor이다(Figure 2).

Conversational Interface. 사용자는 workflow requirement를 자연어로 설명한다. 각 agent turn 전에 현재 pipeline state와 DataFlow operator registry가 MCP를 통해 Claude Code의 context에 주입된다. DataFlow-Skills의 안내를 받아 Claude Code는 user intent를 해석하고 필요한 workflow modification을 결정하며, 이는 MCP tool call로 표현되어 Data Pipeline Backend에 적용된다.

Visual DAG Editor. graphical editor는 workflow를 directed acyclic graph로 render한다. 사용자는 agent가 제안한 change를 inspect하고, parameter를 조정하며, edge를 다시 연결하거나, operator를 직접 추가 및 제거할 수 있다. 모든 manual edit은 즉시 Data Pipeline Backend에 commit되어, 이후 agent interaction이 explicit re-synchronization 없이 최신 workflow state에서 동작하도록 보장한다.

3.3 MCP Tools Layer

agent가 제안했던 수동으로 이루어졌든, 모든 pipeline change는 Request-Validate-Commit protocol을 통과한다.

State Retrieval. 각 synthesis turn이 시작될 때 agent는 이전 turn 이후의 manual edit을 포함하여 최신 pipeline state를 fetch한다.

Mediated Mutation. DataFlow-Skills의 안내를 받아 Claude Code는 의도한 change를 DataFlow registry의 live metadata에 grounded된 typed, structured mutation으로 표현하는 MCP tool call을 발행한다.

Validation. 시스템은 두 가지 property를 verify한다. 업데이트된 pipeline이 directed acyclic graph로 유지되는지, 그리고 각 operator의 output field schema가 모든 downstream operator의 input schema와 compatible한지이다. 어느 check든 실패한 change는 reject된다.

Validated Commitment. validated change는 backend store에 기록된다. WebSocket notification은 updated state를 connected client에 broadcast하여 authoring modality가 동기화되도록 유지한다.

3.4 DataFlow-Skills

DataFlow-Skills는 domain-specific knowledge를 Claude Code의 reasoning context에 주입하여 workflow synthesis를 위한 procedural guidance를 제공한다. MCP Tools Layer는 operator metadata와 workflow state를 노출하지만, 권장 construction strategy나 operator-composition best practice를 encode하지 않는다. 그 결과 agent는 부적절한 operator를 선택하거나, prerequisite processing step을 누락하거나, structurally valid하지만 semantically incorrect한 workflow를 구성할 수 있다.

이를 해결하기 위해 DataFlow-Skills는 두 종류의 지식을 encode한다. 첫 번째는 schema inference, operator selection, parameter configuration, serving verification을 포함하여 권장 workflow-construction sequence를 정의하는 procedural blueprint로 구성된다. 두 번째는 nested structure에 대한 modality matching과 field-flow convention 같은 operator compatibility rule을 포착하는 compositional constraint로 구성된다. 함께, DataFlow-Skills는 agent reasoning을 안내하고, MCP Tools Layer는 live DataFlow environment에 대해 execution을 grounding한다.

4 실험

우리는 DataFlow-Harness를 평가하여 다음 research question에 답한다. 이 질문들은 system의 effectiveness, efficiency, 그리고 underlying mechanism을 점진적으로 보여주도록 구성되어 있다.

RQ1 (Workflow Synthesis Effectiveness). DataFlow-Harness의 native DAG synthesis는 industrial data-processing task에서 execution reliability와 end-to-end task success를 유지하는 측면에서 전통적인 free-form code generation(일회용 script)과 어떻게 비교되는가?

RQ2 (System Efficiency). context-heavy script generation에서 우리의 structured DAG synthesis로 전환함으로써 도입되는 구체적인 computational 및 economic advantage(예: token consumption, latency, API cost)는 무엇인가?

RQ3 (Ablation & Micro-mechanisms). DataFlow-Skills는 task complexity 수준이 달라질 때 pure tool-specification grounding(MCP-only)에 비해 어떻게 개선을 제공하는가?

RQ4 (Downstream Data Quality). governability와 execution reliability를 넘어, agent를 DataFlow-Harness로 grounding하면 해당 pipeline이 생성한 데이터로 학습한 모델의 downstream accuracy로 측정했을 때 higher-quality synthesis pipeline을 작성하게 되는가?

4.1 실험 설정

Benchmark. 우리는 여섯 가지 대표적인 industrial data-processing scenario, 즉 QA generation, review governance, long-document processing, multi-field scoring, schema normalization, low-quality filtering에 걸친 12개 task의 benchmark에서 pipeline-construction capability를 평가한다. 각 task는 input data sample 및 task-specific acceptance criteria와 함께 natural-language objective를 지정한다.

Experimental Settings. 서로 다른 system component의 contribution을 characterize하기 위해, 우리는 platform grounding 수준이 다른 네 가지 agent configuration을 비교한다. (1) **Vanilla CC**: standard Claude Code를 활용하는 unconstrained coding baseline이다. platform-specific context에 access하지 못한 채 disposable Python script를 생성하기 위해 internal parametric knowledge에 전적으로 의존한다. (2) **Context-Aware CC**: agent에게 raw DataFlow codebase가 제공되는 repository-grounded baseline이다. in-context code comprehension을 통해 platform operator를 정확히 mimic할 수 있지만, 본질적으로 관리 불가능한 one-off script를 생성한다. (3) **MCP-Only**: platform-native DAG를 synthesize하도록 엄격히 제약된 tool-augmented baseline이다. DataFlow MCP tool을 활용하여 operator를 동적으로 discover하지만, complex assembly를 위한 explicit procedural guidance가 없다. (4) **DataFlow-Harness**: MCP-based platform grounding과 DataFlow-Skills를 결합하여 editable하고 governable한 workflow DAG를 효율적으로 synthesize하는 우리의 complete framework.

모든 실험은 추론 모델을 고정하기 위해 기본 대규모 언어 모델로 Claude Opus 4.7을 사용한다. agent 동작과 LLM 생성의 확률성을 고려하기 위해, 각 task는 모든 설정에서 10회 실행되며, 그 결과 방법별로 120회의 task 실행이 수행된다.

구성은 의도적으로 서로 다른 action space를 노출한다. script baseline은 임의의 Python을 생성할 수 있는 반면, MCP-only와 DataFlow-Harness는 DataFlow에서 사용할 수 있는 operator를 선택하고 조합한다. 따라서 이 비교는 모델 능력의 고립된 차이가 아니라, 자유 형식 script 생성과 platform 제약 workflow 구성 사이의 end-to-end system trade-off를 측정한다.

평가 Metrics. 제안한 framework를 task success, efficiency, platform integration이라는 세 가지 상호 보완적 차원에서 평가한다.

Task Success. 우리는 **End-to-End (E2E) Pass**를 측정한다. 이는 생성된 workflow가 성공적으로 실행되고 task 별 acceptance criteria를 만족하는 출력을 생성할 것을 요구한다. 이 metric은 workflow 합성, operator 구성, 실행 정확성, 최종 출력 유효성을 포함한 전반적인 workflow 품질을 포착한다.

Efficiency Metrics. 실제 deployment cost를 평가하기 위해 token consumption, monetary cost, workflow construction latency도 추가로 측정한다. (1) **Token Consumption**은 workflow 생성 중 사용된 input 및 output token의 총 수를 보고한다. (2) **Cost**는 기본 모델의 공식 pricing을 사용해 추정하며, workflow를 완성하는 데 필요한 모든 interaction을 포함한다. (3) **Generation Latency**는 task 제출부터 유효한 workflow artifact 생성까지의 wall-clock time을 측정한다.

4.2 Workflow Synthesis 효과 (RQ1)

먼저 자유 형식 code generation의 성능을 특성화한다. Table 1에 보인 것처럼, vanilla generation(Vanilla CC)에 execution context(Context-Aware CC)를 보강하면 end-to-end success가 91.7%에서 94.2%로 향상되며, 복잡한 data-processing task에서 procedural context의 중요성을 부각한다. 그러나 두 접근법 모두 platform-native workflow abstraction과는 분리된 monolithic script를 생성한다.

script generation에서 structured DAG synthesis로 전환하면 상당한 challenge가 도입된다. operator specification만 사용하는 경우(MCP-only) end-to-end success가 83.3%로 감소하는데, 이는 workflow constraint만으로도 모델

Method	Artifact Type	Task Success (%)	Efficiency & Cost		
		End-to-End Pass $\uparrow$	Tokens (In/Out) $\downarrow$	Cost (\$) $\downarrow$	Latency (s) $\downarrow$
Vanilla CC	Disposable Script	91.7	153,584 / 2,474	0.950	190.7
Context-Aware CC	Disposable Script	94.2	185,626 / 1,140	0.456	115.9
MCP-only	Native DAG	83.3	100,607 / 1,273	<u>0.321</u>	<u>105.5</u>
DataFlow-Harness	Native DAG	<u>93.3</u>	74,958 / 891	0.261	95.5

Table 1 end-to-end pass rate를 token usage, monetary cost, generation latency와 함께 보고한다. pass rate는 120회의 task 실행 (12개 task $\times$ 10회 trial)에 대해 계산하며, efficiency metric은 동일한 실행들에 대해 평균을 낸다. **최고 결과는 굵게 표시하고, 두 번째로 좋은 결과에는 밑줄을 그었다.**

에 상당한 reasoning burden을 부과함을 나타낸다. 이 결과는 명확한 *NL2Pipeline gap*을 드러낸다. deployable workflow를 직접 생성하는 것은 executable script를 생성하는 것보다 훨씬 어렵다.

DataFlow-Harness는 명시적인 procedural guidance를 통해 이 gap의 상당 부분을 좁힌다. 이는 93.3%의 end-to-end success를 달성하여 MCP-only보다 10.0 percentage point 향상되며, Context-Aware CC와는 0.9 percentage point 이내에 머문다. 관찰된 pass rate는 수치적으로 가깝지만, 통계적 동등성을 주장하지는 않는다. 이러한 결과는 structured workflow synthesis가 이 benchmark에서 script-generation baseline에 근접하는 신뢰도로 platform-native DAG를 생성할 수 있음을 시사한다.

4.3 Efficiency와 System Cost (RQ2)

Table 1은 DataFlow-Harness가 자유 형식 code generation보다 상당한 efficiency gain을 제공함을 보여준다. Vanilla CC와 비교하면 monetary cost를 72.5%(\$0.950에서 \$0.261로), generation latency를 49.9%(190.7s에서 95.5s로) 줄이면서도 더 높은 end-to-end success rate를 달성한다. 이러한 결과는 structured workflow synthesis가 executable script 생성보다 훨씬 더 resource-efficient함을 나타낸다.

특히, 더 강한 Context-Aware CC baseline과 비교해도 efficiency gain은 유지된다. 거의 동일한 end-to-end performance를 달성하면서도, DataFlow-Harness는 cost를 42.8%, latency를 17.6% 줄여 훨씬 더 유리한 efficiency-performance tradeoff를 보인다.

이 개선은 주로 더 낮은 token consumption에서 비롯된다. script generation에서 native DAG synthesis로 이동하면 MCP-only는 Context-Aware CC 대비 input token usage를 거의 절반으로 줄이며, DataFlow-Harness는 MCP-only와 비교해 total token consumption을 추가로 25.5% 줄인다. 이는 workflow representation이 executable code보다 훨씬 compact하며, procedural guidance가 제약된 operator space 내에서 그 구성을 더욱 streamline한다는 점을 시사한다.

4.4 Textbook-to-VQA Workflow 평가

우리는 또한 도전적인 **textbook-to-VQA extraction** task에서 시스템을 추가로 평가한다. 이 task는 장문 textbook, interleaved solution manual, exam answer sheet를 포함한 heterogeneous educational document에서 question-answer pair를 구성해야 한다. 이 설정은 (i) question과 answer 사이의 long-range dependency, (ii) figure와 table에 대한 visually grounded reasoning, (iii) local textual continuity를 깨뜨리는 highly non-linear document layout 때문에 특히 어렵다.

extraction quality를 포괄적으로 평가하기 위해 FlipVQA-Miner의 [24] 설정을 따르며, 두 가지 상호 보완적 metric을 보고한다. **Precision**은 추출된 QA pair의 정확성을 측정하는 반면, **Coverage Rate**는 source document에서 추출 가능한 QA pair 중 성공적으로 복원된 비율을 측정한다.

Table 2는 document parsing, layout analysis, multimodal content extraction, question-answer alignment, dataset construction을 하나의 workflow로 조합해야 하는 도전적인 textbook-to-VQA extraction task를 평가한다. 기존

Method	Precision ↑	Coverage Rate ↑
Vanilla CC	0.621	0.533
Context-Aware CC	0.893	0.801
MCP-only	0.784	0.621
DataFlow-Harness	0.972	0.873

Table 2 Textbook-to-VQA extraction 성능. Coverage는 document에서 추출 가능한 QA pair 중 성공적으로 복원된 비율을 측정한다.

workflow synthesis benchmark와 비교할 때, 이 설정에서의 성공은 reasoning ability뿐만 아니라 전문화된 document-processing component의 효과적 활용에도 달려 있다.

관찰된 결과는 extraction correctness와 completeness 모두에서 DataFlow-Harness에 유리하며, precision 97.2%와 coverage 87.3%를 보인다. 가장 큰 절대적 개선은 coverage에서 관찰되는데, 여기서 DataFlow-Harness는 baseline 보다 더 많은 유효한 QA pair를 복원한다. 이 pattern은 framework가 단순히 output을 보수적으로 filtering하는 것이 아니라 더 complete한 workflow를 구성함을 시사한다. 이 결과의 일반성을 확인하려면 repeated run과 완전히 명시된 annotation protocol이 필요하다.

이 개선은 부분적으로 DataFlow가 제공하는 풍부한 operator ecosystem을 반영할 수 있다. Textbook-to-VQA extraction에는 PDF parsing, layout recovery, OCR, figure extraction, multimodal understanding, long-range question-answer matching과 같은 capability가 필요하다. 이러한 capability는 재사용 가능한 platform operator로 이미 존재 하지만, 이를 효과적으로 발견하고 조합하는 것은 general-purpose coding agent에게 여전히 어렵다. MCP-only와 DataFlow-Harness 사이의 performance gap은 operator exposure만으로는 충분하지 않으며, workflow construction을 안내하고 관련 operator가 유효한 end-to-end pipeline으로 조립되도록 보장하기 위해 procedural knowledge가 필요함을 나타낸다.

더 넓게 보면, 이 case study는 DataFlow-Harness의 중요한 속성을 부각한다. 그 advantage는 더 강한 model reasoning에서 비롯되는 것이 아니라, 성숙한 platform asset의 systematic reuse를 가능하게 하는 데서 나온다. workflow complexity가 증가할수록, 성공적인 execution은 기능을 처음부터 synthesizing하는 것보다 기존 operator ecosystem을 leveraging하는 데 점점 더 의존한다. 따라서 결과는 현실적인 data-processing scenario에서 NL2Pipeline gap을 좁히는 데 procedural guidance와 platform-native abstraction이 중요하다는 구체적 evidence를 제공한다.

4.5 Ablation과 Micro-mechanisms (RQ3)

앞서 DataFlow-Harness가 native DAG 합성과 자유 형식 code generation 사이의 performance gap을 대체로 해소함을 보였으므로, 다음으로 명시적 절차 guidance가 언제 가장 유익한지 살펴본다. Table 3의 task별 결과는 세 가지 일관된 패턴을 보여준다.

절차 guidance는 task 성공이 암묵적 domain knowledge에 의존할 때 가장 가치 있다. 가장 큰 개선은 QA-generation 및 language-processing tasks(1a, 1b, 3b)에서 나타나며, 이 경우 성공적인 구성에는 호환되는 operator를 선택하는 것 이상의 것이 필요하다. MCP-only는 구조적으로 유효한 DAG를 자주 생성하지만, operator 설명만으로 task-specific procedures를 추론하는 데 어려움을 겪는다. 이러한 procedures를 재사용 가능한 skills로 encoding하면 이 그룹의 aggregate success가 18/30에서 29/30 runs로 향상되어, 전체 이득의 대부분을 설명한다.

절차 guidance는 workflow routing이 straightforward할 때 제한적인 이점만 제공한다. 간단한 transformation 및 filtering tasks(5a, 5b, 6a, 6b)에서는 두 방법이 모두 완전한 성공을 달성한다. 이런 경우에는 operator specifications만으로 workflow structure를 식별하기에 충분하므로, higher-level guidance가 개선할 여지가 거의 없다.

Task	MCP-only	DataFlow-Harness
<u>절차 지식에 의존하는 tasks</u>		
1a: QA basic	6/10	10/10
1b: QA with filter	6/10	9/10
3b: Text-to-QA chain	6/10	10/10
<u>자명하게 routing 가능한 tasks</u>		
5a: Field rename	10/10	10/10
5b: Nested flatten	10/10	10/10
6a: Length filter	10/10	10/10
6b: LLM semantic filter	10/10	10/10
<u>비합성 병목이 있는 tasks</u>		
4a: Score and filter	7/10	7/10
4b: Multi-dimensional score	7/10	7/10
2a: Sentiment	9/10	10/10
2b: Review governance	10/10	9/10
3a: Long-document summary	9/10	10/10

Table 3 10번의 독립 시행에서 측정한 task 별 end-to-end pass count. Tasks는 RQ3에서 논의한 메커니즘에 따라 묶었다.

절차 *guidance*는 *workflow synthesis* 외부의 한계를 극복할 수 없다. 나머지 실패는 주로 *workflow construction* 과 무관한 요인에서 비롯되는 것으로 보인다. 두 방법은 multi-field scoring tasks(4a, 4b)에서 동일한 failure rate를 보이는데, 여기서는 올바른 DAG generation에도 불구하고 outputs가 때때로 downstream numerical constraints를 위반한다. Tasks 2a, 2b, 3a에서는 차이가 작고 때로는 MCP-only에 유리하여, 여러 execution strategies가 유효할 때 prescriptive procedures가 flexibility를 줄일 수 있음을 시사한다.

전반적으로 이 결과는 DataFlow-Skills가 주로 operator specifications만으로는 회복하기 어려운 procedural knowledge를 주입함으로써 기여한다는 점을 보여준다. 관찰된 이점은 모호하고 절차적으로 복잡한 tasks에서 가장 크며, workflow construction이 trivial하거나 underlying model에 의해 bottleneck될 때 줄어든다. 이 ablation은 MCP-only와 full system을 비교하므로, validation의 기여를 별도로 식별하지는 못한다.

4.6 Downstream Training Utility Evaluation (RQ4)

앞선 실험들은 DataFlow-Harness가 governable하고 안정적으로 실행되는 workflows를 생성하는지 평가한다. 별개의, 그리고 논쟁의 여지는 있지만 더 중대한 질문은 harness가 agent가 더 나은 pipelines, 즉 output data가 downstream에서 더 유용한 pipelines를 작성하도록 돕는지이다. 이에 답하기 위해 우리는 end-to-end, outcome-based protocol을 채택한다. 즉 coding agent가 전체 data-synthesis pipeline을 구성하게 하고, pipeline을 실행해 training set을 합성하며, 그 set으로 model을 fine-tune하고, 결과 model의 benchmark accuracy를 측정한다. Pipeline은 궁극적으로 training data를 생성하기 위한 수단이므로, 그것이 내보내는 data의 품질은 간접적이기는 하지만 pipeline quality의 직접적인 척도이다.

Protocol. 각 scenario에서 우리는 정확히 한 가지 점만 다른 두 configurations를 비교한다. **Vanilla CC**에서는 Claude Code가 natural-language task description만 받고 자신의 parametric knowledge로 synthesis pipeline을 작성한다. **DataFlow-Harness**에서는 동일한 agent가 동일한 prompt를 받되, 추가로 DataFlow-Skills와 MCP operator registry를 통해 DataFlow environment에 grounded된다. Pipeline construction 이후의 모든 것은 두 arms에서 고정한다. 즉 동일한 underlying LLMs와 OpenAI-style API settings(concurrency 및 timeout)를 사용해 data를 합성하고, 동일한 수의 samples를 생성하며, 동일한 base model, fine-tuning recipe, evaluation harness를 사용한다. 이 controlled setup은 model, data scale, training settings를 고정한 채 agent-authored pipeline을 downstream accuracy 차이의 주된 원천으로 분리한다.

Table 4 Downstream training으로 본 math pipeline quality. 두 pipeline은 Claude Code가 동일한 natural-language prompt(Prompt 1)에서 작성하며, 동일한 models와 API settings로 data를 합성하고, 동일한 LIMO recipe로 Qwen2.5-32B-Instruct를 fine-tune하는 데 사용된다. DataFlow [16] Table 5 math suite에서 accuracy(%)를 보고하며, AIME24/25는 avg@32로 보고한다. Matched epoch에서 DataFlow-Harness로 생성한 data는 더 높은 average를 산출하여 더 높은 quality의 pipeline임을 시사한다. Epoch별 최고 average는 **bold**로 표시했다.

Pipeline (author)	GSM8K	MATH	AMC23	Olympiad	Gaokao24_mix	Minerva	AIME24@32	AIME25@32	Avg
Qwen2.5-32B-Instruct (base)	95.8	73.5	70.0	38.5	42.9	26.5	16.8	11.6	46.95
1 epoch로 학습									
Vanilla CC	92.3	78.0	47.5	42.8	56.0	35.7	25.1	21.6	49.9
DataFlow-Harness	93.9	72.3	72.5	38.7	38.5	26.5	35.9	34.5	51.6
2 epochs로 학습									
Vanilla CC	94.8	84.0	60.0	48.0	53.8	39.7	31.8	24.3	54.5
DataFlow-Harness	94.4	76.6	75.0	45.2	42.9	25.7	45.4	40.0	55.7

Math Reasoning Pipeline. 첫 번째 scenario(Prompt 1)는 agent에게 math data cleaning-and-synthesis pipeline을 구축하도록 요청한다. incoming problems를 verify하고 filter하며, ill-posed items를 버리고, 각 seed question을 두 개의 새로운 synthetic problems로 확장하며, 모든 question에 대한 reasoning traces를 생성하고, resulting QA pairs에 n -gram deduplication을 적용하되, stage별 models는 별도로 지정한다. 우리는 각 agent-authored pipeline을 동일한 seed pool에서 실행하고, LIMO recipe [27](full-parameter SFT, lr $5e-6$, warmup 없는 cosine schedule, batch size 64, 16K context)에 따라 Qwen2.5-32B-Instruct를 fine-tune한 뒤, DataFlow [16]의 math suite 및 protocol(Table 5)에서 평가하며 AIME24/25는 avg@32로 보고한다.

Table 4는 matched training budgets에서의 결과를 보고한다. 두 configurations 모두 base model을 상당히 끌어올려, Claude Code가 도움 없이도 functional synthesis pipeline을 작성할 수 있음을 확인해 준다. 그러나 모든 matched epoch에서 DataFlow-Harness하에 생성된 data는 더 높은 average accuracy를 산출하며, one epoch 후 49.9에서 51.6으로, two epochs 후 54.5에서 55.7로 개선된다. 이 이득은 가장 어렵고 contamination-sensitive한 benchmarks에 집중된다. DataFlow-Harness data는 one epoch에서 AIME24@32를 25.1에서 35.9로, AIME25@32를 21.6에서 34.5로 높이며, two epochs에서도 유사한 margin을 보인다. 이 패턴은 grounded pipeline이 더 효과적인 verification, filtering, deduplication을 적용하여, 단지 더 많은 reasoning traces가 아니라 더 깨끗하고 더 도전적인 reasoning traces를 생성한다는 해석과 일치한다.

General SFT Pipeline. 두 번째 scenario(Prompt 2)는 더 까다롭다. agent는 seed dataset 없이, API calls를 통해 end-to-end로 data를 생성하는 from-scratch general instruction-tuning pipeline을 구축해야 한다. Pipeline은 세 stages에 걸친다. 여러 difficulty levels를 가진 preset knowledge taxonomy로부터 topic-conditioned 방식으로 다양한 instruction-response pairs를 생성하는 단계, critique-then-rewrite refinement pass, 그리고 low-quality samples를 걸러내는 LLM-as-judge scoring stage이다. 각 agent-authored pipeline은 10K instruction-response pairs를 합성하며, 우리는 이를 사용해 동일한 recipe($8 \times H20$ 에서 LLaMA-Factory와 DeepSpeed ZeRO-3를 사용한 full-parameter SFT, lr $1e-5$, cosine schedule, warmup 0.03, 3 epochs, global batch 128, cutoff 4096, bf16, seed 42)하에 Qwen2.5-7B-Base를 fine-tune한다. 우리는 lm-evaluation-harness [8]로 knowledge(MMLU), math(GSM8K, MATH, Minerva, Olympiad), code(HumanEval, MBPP 및 그 EvalPlus [17] variants) benchmarks 전반에서 fine-tuned models를 평가한다.

Table 5 Downstream training으로 본 general SFT pipeline quality. 두 pipeline은 Claude Code가 동일한 from-scratch synthesis prompt(Prompt 2)에서 작성하며, 동일한 models와 API settings로 10K instruction-response pairs를 생성한다. 동일한 recipe(full-parameter SFT, LLaMA-Factory + DeepSpeed ZeRO-3, 8×H20, lr 1e−5, cosine, warmup 0.03, 3 epochs, global batch 128, cutoff 4096, bf16, seed 42)로 Qwen2.5-7B-Base를 fine-tune하고 lm-evaluation-harness [8]로 평가한다. Code benchmarks는 EvalPlus [17] variants(HE+/MBPP+)를 사용한다. Avg는 9개 benchmark 평균이며, column별 최고값은 **bold**로 표시했다.

Pipeline (author)	Knowledge	Math				Code				Avg
	MMLU	GSM8K	MATH	Minerva	Olympiad	HumanEval	HE+	MBPP	MBPP+	
Vanilla CC	74.4	82.9	68.2	27.6	35.9	78.0	70.1	64.6	51.6	61.5
DataFlow-Harness	74.2	79.5	70.1	27.6	36.3	80.5	72.6	75.4	58.2	63.8

Table 5는 거의 동일한 knowledge performance(MMLU 74.2 vs. 74.4)를 보여주는 한편, 두 pipelines는 개별 math benchmarks에서 서로 승패를 주고받는다. 가장 분명한 차이는 code에서 나타나며, 여기서 DataFlow-Harness pipeline은 네 benchmarks 모두에서 더 강하고, MBPP에서 가장 큰 격차(75.4 vs. 64.6)를 보인다. 이러한 code gains는 전체 nine-benchmark average를 2.3 points 끌어올린다(63.8 vs. 61.5). 이 패턴은 grounded critique-then-rewrite 및 judge stages가 더 executable하고 더 잘 구조화된 coding responses를 생성한다는 해석과 일치한다. 두 scenarios는 함께, grounding이 agent-authored pipeline이 생성한 data의 utility를 향상할 수 있다는 preliminary outcome-level evidence를 제공한다. 각 scenario는 independently authored pipelines와 multiple training seeds 전반의 repeated experiment가 아니라 controlled case study이므로, 이 결과를 general causal estimate로 해석해서는 안 된다.

5 Conclusion

우리는 procedural Skills, live MCP grounding, typed mutations, structural validation, 그리고 synchronized conversational and visual editing을 결합하여 *NL2Pipeline gap*을 다루는 platform인 DataFlow-Harness를 제시했다. 우리의 12-task benchmark에서 관찰된 end-to-end pass rate는 script-generation baselines에 가깝고, 측정된 construction cost와 latency는 더 낮다. Task별 ablation은 절차 guidance가 어디에서 가장 유용한지도 추가로 보여준다.

Limitations. 우리 평가는 하나의 coding-agent 및 model family와 비교적 작은 platform-specific benchmark를 사용한다. 사용 가능한 ablation은 모든 component를 분리하지 못하며, schema validation은 semantic correctness를 보장할 수 없다. 우리는 task-clustered confidence intervals나 사전에 명시한 non-inferiority test 없이 관찰된 averages를 보고한다. 또한 cost reporting은 prompt caching을 사용할 때 독립적으로 재계산되려면 token-class breakdown이 필요하다. 마지막으로 downstream-utility 결과는 multiple independently authored pipelines와 training seeds 없이 두 case studies만 다룬다. 더 넓은 workflow-governance claims를 제기하기 전에 persistence, reuse, provenance, concurrent editing, recovery에 대한 직접 평가가 필요하다.

References

- [1] Anthropic. Claude Code: An agentic command-line coding assistant. <https://docs.anthropic.com/claude/docs/claude-code>, 2024. Accessed: 2026-05-07.
- [2] Anthropic. Model Context Protocol: An open standard for connecting AI assistants to data and tools. <https://modelcontextprotocol.io>, 2024. Accessed: 2026-05-07.
- [3] Balis et al. From research question to scientific workflow: Leveraging agentic ai for science automation. *arXiv preprint arXiv:2604.21910*, 2026. URL <https://arxiv.org/abs/2604.21910>.
- [4] Daoyuan Chen, Yilun Huang, Zhijian Ma, Hesun Chen, Xuchen Pan, Ce Ge, Dawei Gao, Yuexiang Xie, Zhaoyang Liu, Jinyang Gao, et al. Data-juicer: A one-stop data processing system for large language models, 2023. URL <https://arxiv.org/abs/2309.02033>.

- [5] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Pondé de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code, 2021. URL <https://arxiv.org/abs/2107.03374>.
- [6] Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. Teaching large language models to self-debug. In The Twelfth International Conference on Learning Representations (ICLR 2024), 2024. URL <https://openreview.net/forum?id=KuPixIqPiq>.
- [7] Matthias Galster, Seyedmoein Mohsenimofidi, Jai Lal Lulla, Muhammad Auwal Abubakar, Christoph Treude, and Sebastian Baltes. Configuring agentic ai coding tools: An exploratory study. arXiv preprint arXiv:2602.14690, 2026.
- [8] Leo Gao, Jonathan Tow, Baber Abbasi, et al. A framework for few-shot language model evaluation, 2024. URL <https://zenodo.org/records/12608602>.
- [9] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yixin Dai, Jiawei Sun, Haofen Wang, Haofen Wang, et al. Retrieval-augmented generation for large language models: A survey. arXiv preprint arXiv:2312.10997, 2(1):32, 2023.
- [10] Sirui Hong, Yizhang Lin, Bang Liu, Bangbang Liu, Binhao Wu, Ceyao Zhang, Danyang Li, Jiaqi Chen, Jiayi Zhang, Jinlin Wang, et al. Data interpreter: An llm agent for data science. In Findings of the Association for Computational Linguistics: ACL 2025, pages 19796–19821, 2025.
- [11] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. DSPy: Compiling declarative language model calls into self-improving pipelines. In The Twelfth International Conference on Learning Representations (ICLR 2024), 2024. URL <https://openreview.net/forum?id=sY5N0zY50d>.
- [12] Dominik Kreuzberger, Niklas Kühl, and Sebastian Hirschl. Machine learning operations (mlops): Overview, definition, and architecture. IEEE access, 11:31866–31879, 2023.
- [13] Li et al. Autoflow: Automated workflow generation for large language model agents. arXiv preprint arXiv:2407.12821, 2024. URL <https://arxiv.org/abs/2407.12821>.
- [14] Jeffrey Li, Alex Fang, Georgios Smyrnis, Maor Ivgi, Matt Jordan, Samir Yitzhak Gadre, Hritik Bansal, Etash Guha, Sedrick Scott Keh, Kushal Arora, Saurabh Garg, Rui Xin, Niklas Muennighoff, Reinhard Heckel, Jean Mercat, Mayee F. Chen, Suchin Gururangan, Mitchell Wortsman, Alon Albalak, Yonatan Bitton, Marianna Nezhurina, Amro Abbas, Cheng-Yu Hsieh, Dhruva Ghosh, Josh Gardner, Maciej Kilian, Hanlin Zhang, Rulin Shao, Sarah M. Pratt, Sunny Sanyal, Gabriel Ilharco, Giannis Daras, Kalyani Marathe, Aaron Gokaslan, Jieyu Zhang, Khyathi Raghavi Chandu, Thao Nguyen, Igor Vasiljevic, Sham Kakade, Shuran Song, Sujay Sanghavi, Fartash Faghri, Sewoong Oh, Luke Zettlemoyer, Kyle Lo, Alaaeldin El-Nouby, Hadi Pouransari, Alexander Toshev, Stephanie Wang, Dirk Groeneveld, Luca Soldaini, Pang Wei Koh, Jenia Jitsev, Thomas Kollar, Alexandros G. Dimakis, Yair Carmon, Achal Dave, Ludwig Schmidt, and Vaishaal Shankar. DataComp-LM: In search of the next generation of training sets for language models. In Advances in Neural Information Processing Systems 38 (NeurIPS 2024), 2024. URL <https://arxiv.org/abs/2406.11794>.
- [15] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, Qian Liu, Evgenii Zheltonozhskii, Terry Yue Zhuo, Thomas Wang, Olivier Dehaene, Mishig Davaadorj, Joel Lamy-Poirier, João Monteiro, Oleh Shliazhko, Nicolas Gontier, Nicholas Meade, Armel Zebaze, Ming-Ho Yee, Logesh Kumar Umapathi, Jian Zhu, Benjamin Lipkin, Muhtasham Oblokulov, Zhiruo Wang, Rudra Murthy V, Jason T. Stillerman, Siva Sankalp Patel, Dmitry Abulkhanov, Marco Zocca, Manan Dey, Zhihan Zhang, Nour Fahmy, Urvasi Bhattacharyya, Wenhao Yu, Swayam Singh, Sasha Luccioni, Paulo Villegas, Maxim Kunakov, Fedor Zhdanov, Manuel Romero, Tony Lee, Nadav Timor, Jennifer Ding, Claire Schlesinger, Hailey Schoelkopf, Jan Ebert, Tri Dao, Mayank Mishra, Alex Gu, Jennifer Robinson, Carolyn Jane Anderson, Brendan Dolan-Gavitt, Danish Contractor, Siva Reddy, Daniel Fried, Dzmitry Bahdanau, Yacine Jernite, Carlos Muñoz Ferrandis, Sean Hughes, Thomas Wolf, Arjun Guha, Leandro von Werra, and Harm de Vries. StarCoder: may the source be with you! Transactions on Machine Learning Research, 2023. URL <https://openreview.net/forum?id=Kof0g41haE>.

- [16] Zhiyuan Liang et al. DataFlow: An LLM-driven framework for unified data preparation and workflow automation in the era of data-centric AI, 2025. URL <https://arxiv.org/abs/2512.16676>.
- [17] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation. Advances in Neural Information Processing Systems (NeurIPS), 2023.
- [18] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. Agentbench: Evaluating llms as agents. In International Conference on Learning Representations, volume 2024, pages 52989–53046, 2024.
- [19] Shishir G Patil, Tianjun Zhang, Xin Wang, and Joseph E Gonzalez. Gorilla: Large language model connected with massive apis. Advances in Neural Information Processing Systems, 37:126544–126565, 2024.
- [20] Bo Qiao, Liqun Li, Xu Zhang, Shilin He, Yu Kang, Chaoyun Zhang, Fangkai Yang, Hang Dong, Jue Zhang, Lu Wang, et al. Taskweaver: A code-first agent framework. arXiv preprint arXiv:2311.17541, 2023.
- [21] Yujia Qin, Shengding Hu, Yankai Lin, Weize Chen, Ning Ding, Ganqu Cui, Zheni Zeng, Xuanhe Zhou, Yufei Huang, Chaojun Xiao, et al. Tool learning with foundation models. ACM Computing Surveys, 57(4):1–40, 2024.
- [22] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. Toolllm: Facilitating large language models to master 16000+ real-world apis. In International Conference on Learning Representations, volume 2024, pages 9695–9717, 2024.
- [23] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In Advances in Neural Information Processing Systems, volume 36, 2023. URL <https://arxiv.org/abs/2303.11366>.
- [24] Zhen Hao Wong, Jingwen Deng, Hao Liang, Runming He, Chengyu Shen, and Wentao Zhang. Flipvqa-miner: Cross-page visual question-answer mining from textbooks. arXiv preprint arXiv:2511.16216, 2025.
- [25] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. Autogen: Enabling next-gen llm applications via multi-agent conversations. In First conference on language modeling, 2024.
- [26] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering, 2024. URL <https://arxiv.org/abs/2405.15793>.
- [27] Yixin Ye, Zhen Huang, Yang Xiao, Ethan Chern, Shijie Xia, and Pengfei Liu. LIMO: Less is more for reasoning. arXiv preprint arXiv:2502.03387, 2025.
- [28] Daochen Zha, Zaid Pervaiz Bhat, Kwei-Herng Lai, Fan Yang, Zhimeng Jiang, Shaochen Zhong, and Xia Hu. Data-centric artificial intelligence: A survey. arXiv preprint arXiv:2303.10158, 2023. URL <https://arxiv.org/abs/2303.10158>.